

Exploration Sizing via Model-Predictive Control

Ethan Che*
Meta
New York, NY, USA
eche@meta.com

James McInerney
Netflix
Los Gatos, CA, USA
jmcinerney@netflix.com

Hakan Ceylan†
Amazon
San Francisco, CA, USA
hakancey@amazon.com

Nathan Kallus
Netflix
Los Gatos, CA, USA
Cornell University
New York, NY, USA
kallus@cornell.edu

Abstract

Modern recommendation systems rely on exploration to learn user preferences for new items, typically implementing uniform exploration policies (e.g., epsilon-greedy) due to their simplicity and compatibility with machine learning (ML) personalization models. Within these systems, a crucial consideration is the rate of exploration - what fraction of user traffic should receive random item recommendations and how this should evolve over time. While various heuristics exist for navigating the resulting exploration-exploitation tradeoff, selecting optimal exploration rates is complicated by practical constraints including batched updates, time-varying user traffic, and short time horizons. In this work, we propose a principled framework for determining the exploration schedule based on directly minimizing Bayesian regret through stochastic gradient descent (SGD), allowing for dynamic exploration rate adjustment via Model-Predictive Control (MPC). Through extensive experiments with recommendation datasets, we demonstrate that variations in the batch size across periods significantly influence the optimal exploration strategy. Our optimization methods automatically calibrate exploration to the specific problem setting, consistently matching or outperforming the best heuristic for each setting.

CCS Concepts

• **Theory of computation** → **Sequential decision making**; • **Information systems** → **Personalization**.

Keywords

Contextual bandits, personalization, exploration.

ACM Reference Format:

Ethan Che, Hakan Ceylan, James McInerney, and Nathan Kallus. 2026. Exploration Sizing via Model-Predictive Control. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3774904.3792661>

*Corresponding author.

†Work done while at Netflix.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792661>

1 Introduction

Modern recommendation systems apply sophisticated machine learning (ML) algorithms to historical interaction data to determine optimal personalized recommendations for each user. This shift has substantially improved recommendation quality but introduces significant challenges when new items or treatments enter the system. The introduction of new items creates the well-documented "cold-start" problem in recommendation systems [37], where no historical data exists to inform personalization decisions. While numerous techniques have been proposed for extrapolating from existing items to new ones using content features or collaborative signals, a prevalent practice remains collecting new interaction data through dedicated exploration phases [6, 25, 30]. This approach creates a fundamental tension: more exploration yields better training data for recommendation algorithms but delays the delivery of optimally personalized experiences to users. This tension represents the classic "exploration-exploitation" trade-off extensively studied in both recommendation systems and the broader bandit literature [1, 24].

Although sophisticated algorithms from the contextual bandit literature such as Thompson Sampling (TS) [36] are theoretically shown to be effective at managing this trade-off, production recommendation systems often implement simpler exploration strategies based on *uniform exploration* of items due to (1) engineering and operational constraints [29] and (2) compatibility with black-box machine learning models for personalization. Two particularly common approaches in recommendation platforms are:

- **Explore-then-commit (ETC):** [12] This strategy dedicates an initial period to randomly assigning treatments across the user base before switching to a fully personalized policy.
- **Epsilon-greedy:** [40] This approach continuously allocates a fixed percentage of traffic to random items while using the current ML policy for the remainder.

A critical challenge for recommendation system practitioners implementing these strategies is determining the optimal rate of exploration - how many users will be allocated a random item instead of the item recommended by the ML algorithm? For ETC, this means selecting the appropriate duration of the initial exploration phase. For epsilon-greedy, it requires setting the right percentage of traffic devoted to exploration. While theoretical guidelines exist in the

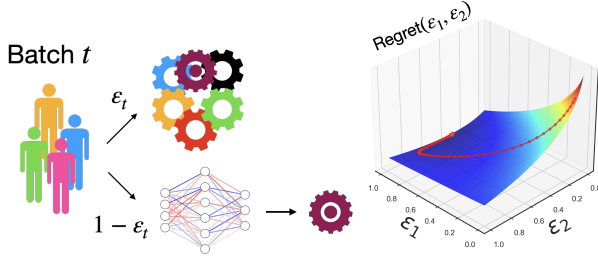


Figure 1: Our work studies how to set the exploration rate for uniform exploration algorithms, for which, in each period t , assign a random item to ε_t -fraction of users and a personalized item to the remaining $1 - \varepsilon_t$ (see left). Our proposed method selects the exploration rates for each period $\varepsilon_t \in [0, 1]$ by optimizing the exploration rates via stochastic gradient descent (SGD) to minimize a differentiable approximation of Bayesian Regret. The right figure displays this approximation as a function of the exploration rate in the first two periods (ε_1 and ε_2). The iterations of SGD (red dots) starting from an initial plan of zero exploration $\varepsilon_1 = \varepsilon_2 = 0$ converge to an exploration plan that decreases the rate of exploration over time $\varepsilon_1 = 1.0, \varepsilon_2 = 0.3$.

bandit literature [24, 43], real-world recommendation systems face additional complexities that theoretical models often overlook:

- **Batched updates:** [42] Unlike the fully sequential sampling assumed in theoretical work, real recommendation systems typically collect data in batches and update models periodically (e.g., daily), introducing latency between exploration and policy improvement [9, 11, 32].
- **Time-varying user traffic:** Recommendation platforms experience fluctuating traffic patterns, especially when new items are released (e.g. new movies). Thus, the batch sizes can vary substantially across time periods, affecting the rate at which exploration data accumulates.
- **Limited horizons:** New content launches or seasonal recommendations often operate within constrained timeframes (1-2 weeks), requiring rapid convergence to optimal policies before user interest wanes.

These operational realities of recommendation systems significantly complicate the exploration strategy. Despite the maturity of the recommendation systems field, there remains a notable gap between theoretical exploration strategies and practical implementation guidance for these constrained, real-world recommendation scenarios.

In this work, we introduce a principled framework for optimizing exploration in limited horizon, batched settings for linear contextual bandits. Our motivating observation is that cumulative regret, the core performance metric for bandit systems, is difficult to optimize directly due to uncertainty over the true environment. Instead, we show that *Bayesian regret* leads to a differentiable objective that is directly amenable to gradient-based optimization. In order to mitigate potential errors from relying on a Bayesian model, we propose *re-optimizing* the exploration schedule after each batch – a control strategy known as model-predictive control (MPC) [22],

which is widely used in domains such as robotics [19] and chemical engineering [33] but to our knowledge, has not been yet been applied to tuning exploration in contextual bandits.

In summary, our contributions are as follows:

- Using a Bayesian model, we obtain a *differentiable* formulation of Bayesian regret as a function of the exploration rates that can be minimized through stochastic gradient descent (SGD).
- By re-solving the optimization problem after every batch—a control policy known as model-predictive control (MPC)—the platform can adjust the exploration rate dynamically.
- We find that our proposed methods are able to adapt the exploration schedule to the structure of the batched arrivals, and match or outperform the best heuristic for each setting.
- We find surprisingly that in these short-horizon, batched settings, uniform exploration with an appropriate exploration schedule can even outperform more sophisticated approaches such as Batched Thompson Sampling [17, 18].

While we focus on the linear-contextual bandit setting, our method can be extended to deep-learning models by utilizing a Bayesian linear model on the last-layer features of the network; also known as the NeuralLinear [35] algorithm which has strong empirical performance in practice [39].

The paper is organized as follows. We discuss related work on exploration and contextual bandits in recommendation systems in Section 2. In Section 3, we introduce the setting and the contextual bandit problem. In Section 4, we describe the Bayesian formulation and the regret objective. In Section 5, we discuss the optimization problem and the model-predictive control algorithm. In Section 6, we benchmark our proposed algorithms with baseline exploration strategies on several datasets. Finally, conclusions and future work are discussed in Section 7.

2 Related Work

Our work is closely related to bandits for personalization and recommendation as well as Bayesian optimal experimental design.

Bandits for Recommendation Systems. Multi-armed and contextual bandits are a popular methodology for cold-start exploration and personalization [1, 6, 13, 28, 29, 38, 41]. Rather than designing a new bandit policy, we focus on existing uniform exploration policies, due to their widespread usage [29], and develop a framework to tune exploration rates to handle practical constraints such as batched/infrequent feedback [42].

Bayesian Optimal Experimental Design. Our methods rely on a Bayesian model to calibrate exploration for personalization, similar to [44]. Our approach, which formulates exploration as an optimization problem, is closely related to Bayesian methods for optimal experimental design [10, 34] and Bayesian methods for regret minimization [4, 5]. However, while typically these works typically focus on information gain or minimizing posterior uncertainty, our objective is Bayesian regret and we devise efficient approximations for optimizing this. Furthermore, our scope is limited to tuning the rate of exploration, rather than experimental selection.

3 Setting

We consider the problem of selecting the exploration rate when collecting data to train a personalized assignment policy on newly-released items over a fixed time horizon. Following convention, we model this as a batched contextual bandit problem, which we proceed to formalize.

We denote $\mathcal{A}$ to be a set of $K \in \mathbb{N}$ newly-released items. There are $T \in \mathbb{N}$ time periods and in each period $t = 1, \dots, T$ a batch of $n_t \in \mathbb{N}$ users arrive to the platform, with a total of $n = \sum_{t=1}^T n_t$ users. For example, each period t could correspond to a day, in which case n_t is number of impressions that arrive on day t . Each user has a user embedding $X_i \in \mathbb{R}^d$, and we assume that in each batch users arrive iid from a distribution $X_i \sim \mu$. We assume that the user embeddings are X_i frozen and are not being learned simultaneously.

In each period t , after the batch of n_t users arrives, the platform assigns each user i a *single* item $A_i \in \mathcal{A}$ according to a policy π_t , which is a probability distribution over $\mathcal{A}$ conditional on the user embedding X_i , i.e. $A_i \sim \pi_t(\cdot | X_i)$. After assigning an item to the user, the platform observes a reward metric R_i , e.g. engagement or user rating. We consider a standard form of the reward model, which posits reward as a linear function of the user embedding:

$$r(x, a) \equiv x^\top \theta_a \quad (1)$$

where $\theta_a \in \mathbb{R}^d$ is an item embedding that is unknown to the platform. As a result, this problem is a linear contextual bandit problem. We refer to $\theta \in \mathbb{R}^{Kd}$ as the vector containing the embeddings for all K items. The platform only observes a noisy reward:

$$R_i \equiv R_i(A_i) = r(X_i, A_i) + \eta_i \quad (2)$$

where η_i are iid random variables with zero mean $\mathbb{E}[\eta_i] = 0$ and variance $\text{Var}(\eta_i) = s^2$. After collecting data $D_t = \{(X_i, A_i, R_i)\}_{i=1}^{n_t}$ from batch t , the platform updates their estimate for θ , denoted as $\hat{\theta}_t$, and their policy π_{t+1} for the next batch.

Crucially, due to the batched nature of the feedback, the platform only updates their policy after observing all the rewards at the end of the batch. This is designed to match practical settings for which the personalization algorithm is updated after observing a large batch of data (e.g. all the interactions during a day), instead of being updated after each sample, as is the standard assumption in the contextual bandit literature.

A commonly considered goal is to find a policy that minimizes expected cumulative regret along the time horizon:

$$\min_{\pi} \text{Regret}(\pi) \equiv \min_{\pi} \sum_{t=1}^T \sum_{i=1}^{n_t} \mathbb{E}_{X_i \sim \mu} [\text{Reg}(X_i, \pi_t)] \quad (3)$$

where the regret for user i is defined to be the difference between the expected reward under the best item and the expected reward achieved under the policy,

$$\text{Reg}(X_i, \pi_t) \equiv r^*(X_i) - \mathbb{E}_{A_i \sim \pi_t(\cdot | X_i)} [r(X_i, A_i)] \quad (4)$$

$$r^*(X_i) \equiv \max_{a \in \mathcal{A}} \{r(X_i, a)\} \quad (5)$$

While there exist policies, such as (batched) Thompson Sampling, that are designed to achieve sublinear regret (3) as the number of users n grows large, we focus on the class of *uniform exploration policies*, motivated by the widespread use in production recommendation systems. Due to the simplicity of their design, uniform

exploration policies are easy to implement, work well with batched feedback and other production constraints, and are compatible with more complex reward models.

At each period t , the policy randomly allocates a fraction of users to a uniformly sampled item. We denote this fraction as the *exploration rate* ε_t and refer to this group as the ‘explore’ group. We let $\xi_i \in \{0, 1\}$ indicate whether i is in the explore group. The remaining $1 - \varepsilon_t$ are assigned an item ‘greedily’ based on the current estimate $\hat{\theta}_t$ of θ (the ‘exploit’ group). As a result, the policy will be of the following form:

$$\pi_t(a | X_i) = \begin{cases} 1/K & \text{w.p. } \varepsilon_t \\ \mathbf{1}\{a = \arg \max_a X_i^\top \hat{\theta}_{t,a}\} & \text{w.p. } 1 - \varepsilon_t \end{cases} \quad (6)$$

Restricting ourselves to policies of this form, a key question is determining the proper exploration rates $\varepsilon_{1:T} \equiv \{\varepsilon_t\}_{t=1}^T$. We can express regret as a function of the exploration rates (abusing notation):

$$\text{Regret}(\varepsilon_{1:T}) \equiv \sum_{t=1}^T \sum_{i=1}^{n_t} \mathbb{E}_{X_i \sim \mu} [\text{Reg}(X_i, \varepsilon_t)] \quad (7)$$

$$\text{Reg}(X_i, \varepsilon_t) \equiv \varepsilon_t \text{Reg}(X_i, 1/K) + (1 - \varepsilon_t) \text{Reg}(X_i, \hat{\pi}_t) \quad (8)$$

$$\hat{\pi}_t(a | X_i) \equiv \mathbf{1}\{a = \arg \max_a X_i^\top \hat{\theta}_{t,a}\} \quad (9)$$

where $\text{Reg}(X_i, 1/K)$ is the regret under a uniformly drawn item and $\text{Reg}(X_i, \hat{\pi}_t)$ is regret under the greedy assignment of the item based on the current estimate $\hat{\theta}_t$.

Two well-known examples of uniform exploration policies are:

- Epsilon-greedy uses a constant exploration rate $\varepsilon_t = \varepsilon$.
- Explore-then-Commit (ETC) sets a fixed budget of B samples before committing to the greedy policy. Thus, $\varepsilon_t = 1$ until the budget is reached after which $\varepsilon_t = 0$:

$$\varepsilon_t = \min \left\{ \frac{(B - \sum_{s=1}^{t-1} \varepsilon_s n_s)}{n_t}, 1 \right\} \quad (10)$$

Note that ε_t may be fractional if $\sum_{s=1}^{t-1} n_s < B < \sum_{s=1}^t n_s$.

While generally a larger exploration rate ε_t will incur more regret in period t , as typically assigning a random item has higher regret $\text{Reg}(X_i, 1/K) > \text{Reg}(X_i, \hat{\pi}_t)$, exploration improves the estimate $\hat{\theta}_t$ for future periods – this presents the classic ‘exploration-exploitation’ trade-off. However, it is not clear how to set the exploration rates in order to precisely optimize this trade-off, especially in batched settings with (potentially) time-varying batch sizes n_t . And while there exist theoretically-derived exploration rates (e.g. in [12, 24]), they often involve either unknown quantities (e.g. gaps between average rewards) or under-specified constants that can impact practical performance. As a result, existing practices for selecting ε_t are typically ad-hoc.

4 Differentiable Bayesian Regret

Ideally, we could select the exploration rates $\varepsilon_{1:T}$ to directly minimize $\text{Regret}_T(\varepsilon_{1:T})$ in (7). However, there are two issues that make $\text{Regret}_T(\varepsilon_{1:T})$ unsuitable for optimization. First, $\text{Regret}_T(\varepsilon_{1:T})$ depends on the unknown item embeddings θ through the reward model $r(x, a)$. Second, quantifying the benefit of exploration requires precisely understanding how additional samples will affect the greedy policy in future periods, which does not have a simple closed form that can be optimized. In general, it is well-known

that regret is a difficult objective to directly optimize, which is why existing algorithmic principles such as UCB or Thompson Sampling rely on optimizing proxies or upper bounds of regret instead (e.g. Hoeffding Bound) [24]. However, in this section, we show that *Bayesian regret* does not suffer from these two issues and is amenable to optimization.

We assume the platform has a prior distribution over the unknown item embeddings:

$$\theta \sim N(\beta_1, \Sigma_1) \quad (11)$$

where $\beta_1 \in \mathbb{R}^{Kd}$ and $\Sigma_1 \in \mathbb{R}^{Kd \times Kd}$ are the prior mean and covariance of the item embeddings. Typically, $\beta_1 = \mathbf{0}$ and $\Sigma = \sigma^2 \mathbf{1}$ for some $\sigma^2 > 0$, although the prior could be calibrated to the statistics of existing item embeddings.

We introduce three assumptions that enable simplify the structure of the Bayesian posterior updates. First, we assume Gaussian noise, as this implies that the posterior distribution of θ is also Gaussian with closed-form expressions for the posterior mean and covariance.

ASSUMPTION 1. *The noise η_i are distributed iid and are normally distributed $N(0, s^2)$.*

We also assume that posterior updates only use data from the ‘explore’ group:

ASSUMPTION 2. *The posterior is updated using only data from the ‘explore’ group, i.e. the history we are conditioning on $H_t = \sigma(\cup_{s=1}^{t-1} \mathcal{D}_s)$ where $\mathcal{D}_s = \cup_{i=1}^{n_s} \{(X_i, A_i, R_i, \xi_i) : \xi_i = 1\}$.*

One limitation of this assumption is that it ignores data gathered by the greedy policy. Nevertheless this captures the exploration-exploitation tradeoff and reflects practical implementations, which typically only use randomly sampled data to train the policy.

Our final assumption is that the prior covariance Σ_1 is block-diagonal across items.

ASSUMPTION 3. *We assume that the prior covariance is of the form:*

$$\Sigma_1 = \text{diag}(\Sigma_{1,1}, \dots, \Sigma_{1,a}, \dots, \Sigma_{K,a})$$

where $\Sigma_{1,a} \in \mathbb{R}^{d \times d}$ for all $a \in \mathcal{A}$.

We do not need this assumption for the validity of Bayesian updating, but it simplifies notation by allowing for the posterior to be updated independently for each item.

Under these assumptions, we obtain a standard expression for the posterior distribution of θ at the beginning of period t . The posterior distribution of θ given H_t follows a normal distribution:

$$\theta|H_t \sim N(\beta_t, \Sigma_t)$$

where the posterior mean β_t and covariance Σ_t follow the recursive formula for each a :

$$\Sigma_{t+1,a} = (\Sigma_{t,a}^{-1} + I_{t,a}(\epsilon_t))^{-1} \quad (12)$$

$$\beta_{t+1,a} = \Sigma_{t+1,a} \left(\Sigma_{t,a}^{-1} \beta_t + I_{t,a}(\epsilon_t) \left(\sum_{i=1}^{n_t} \xi_i \mathbf{1}\{A_i = a\} R_i X_i \right) \right) \quad (13)$$

$$I_{t,a}(\epsilon_t) \equiv s^{-2} \sum_{i=1}^{n_t} \xi_i \mathbf{1}\{A_i = a\} X_i X_i^\top \quad (14)$$

We use the notation $\mathbb{E}_1$ to denote expectation of θ under the prior and $\mathbb{E}_t[\cdot] \equiv \mathbb{E}[\cdot|H_t]$ to denote expectation under the posterior at period t so,

$$\mathbb{E}_t[r(x, a)] = x^\top \mathbb{E}_t[\theta_a] = x^\top \beta_{t,a} \quad (15)$$

An important property of the Bayesian linear regression model is the existence of closed-form expressions for the distribution of *future* posterior states, conditional on the current posterior and a fixed exploration rate. A derivation can be found in [5].

LEMMA 4.1. [5] *Suppose Assumptions 1-3 hold. Consider any exploration rates $\epsilon_{1:T}$, and any $1 \leq t \leq s$. For each item a , the distribution of the posterior $(\beta_{s,a}, \Sigma_{s,a})$, conditional on the period t posterior $(\beta_{t,a}, \Sigma_{t,a})$ is given as follows:*

$$\Sigma_{s,a} \stackrel{d}{=} \left(\Sigma_{t,a}^{-1} + \sum_{l=t}^{s-1} I_{l,a}(\epsilon_l) \right)^{-1} \quad (16)$$

$$\beta_{s,a} \stackrel{d}{=} \beta_{t,a} + (\Sigma_{t,a} - \Sigma_{s,a})^{1/2} Z_s \quad (17)$$

where $Z_t \sim N(0, I)$ is a standard Gaussian random vector and $I_{n,s}^a$ is the empirical design matrix given by (14) with $X_i \stackrel{iid}{\sim} \mu$ (μ is the distribution of user embeddings), $\xi_i \stackrel{iid}{\sim} \text{Ber}(\epsilon_s)$, $A_i \stackrel{iid}{\sim} \text{Multinomial}(\mathbf{1}/K)$.

Lemma 4.1 captures how future posterior beliefs will change based on the exploration rates. Setting a high exploration rate ϵ_t induces a larger update in future beliefs while a low exploration rate will not change future beliefs from the current posterior. Thus, the lemma precisely measures the *value of exploration* not only for the next period but for every future period in the time horizon, and doing so only requires sampling from standard distributions.

We now turn to the Bayesian regret objective, which is more amenable to optimization as it does not depend on unknown item embeddings. Instead, it averages regret (7) over item embeddings drawn from the prior distribution:

$$\min_{\epsilon_{1:T}} \mathbb{E}_{\theta \sim N(\beta_1, \Sigma_1)} [\text{Regret}_T(\epsilon_{1:T})] \quad (18)$$

$$= \min_{\epsilon_{1:T}} \sum_{t=1}^T n_t \mathbb{E}_1[\epsilon_t \text{Reg}(X_i, \mathbf{1}/K) + (1 - \epsilon_t) \text{Reg}(X_i, \hat{\pi}_t)] \quad (19)$$

where the expectation in the second line denotes expectation over $X_i \sim \mu$, $\xi_i \sim \text{Ber}(\epsilon_i)$, random sampling A_i , and the prior distribution of θ . One benefit of the Bayesian model is that Regret can be written in closed form in terms of prior and posterior parameters. To illustrate, we first look at the regret of the greedy policy $\text{Reg}(X_i, \hat{\pi}_t)$. Given the Bayesian formulation, we slightly revise the definition of the greedy policy $\hat{\pi}_t$: it assigns the item that maximizes expected reward at period t ,

$$\begin{aligned} \hat{\pi}_t(a|x) &\equiv \mathbf{1}\{a = \arg \max_a \mathbb{E}_t[r(X_i, a)]\} \\ &= \mathbf{1}\{a = \arg \max_a X_i^\top \beta_{t,a}\}. \end{aligned} \quad (20)$$

We arrive at our main result, which gives a concrete expression for Bayesian regret.

PROPOSITION 4.2. *Under Assumptions 1-3, Bayesian regret (18) under the uniform exploration policy with the greedy assignment*

given by (20) is equal to

$$J_1(\varepsilon_{1:T}, \beta_1, \Sigma_1) \equiv \mathbb{E}_1[\text{Regret}_T(\varepsilon_{1:T})] \quad (21)$$

$$= n\mathbb{E}_1[r^*(X_i)] - \sum_{t=1}^T n_t \mathbb{E} \left[\varepsilon_t X_i^\top \bar{\beta}_1 + (1 - \varepsilon_t) \max_a X_i^\top \beta_{t,a} \right]$$

where $\bar{\beta}_1 = (1/K) \sum_{k=1}^K \beta_{1,a}$ and for all items $a \in \mathcal{A}$,

$$\Sigma_{t,a}^{-1} = \Sigma_{1,a}^{-1} + \sum_{l=1}^{t-1} I_{l,a}(\varepsilon_l) \quad (22)$$

$$\beta_{t,a} = \beta_{1,a} + (\Sigma_{1,a} - \Sigma_{t,a})^{1/2} Z_t \quad (23)$$

where $I_{t,a}(\varepsilon_t)$ is defined in (14) and the expectation in (21) is over $Z_t \stackrel{iid}{\sim} N(0, I)$, $X_i \stackrel{iid}{\sim} \mu$, $\xi_i \stackrel{iid}{\sim} \text{Ber}(\varepsilon_s)$, and $A_i \stackrel{iid}{\sim} \text{Multinomial}(1/K)$.

See proof in Section A. Proposition 4.2 provides an exact expression for Bayesian regret, which can be efficiently estimated through Monte Carlo simulation of standard random variables based on known information. It presents a precise characterization of the exploration-exploitation trade-off – a higher ε_t reduces immediate reward by placing more weight on randomization, but increases future reward by increasing the variance of future posteriors $\beta_{t,a}$ (i.e. inducing a larger posterior update), which increases the expected reward from the greedy policy in the future $\mathbb{E}_1[\max_a X_i^\top \beta_{t,a}]$.

Still, the expressions (22)-(23) are difficult to directly optimize because $I_{t,a}(\varepsilon_t)$ depends on ε_t through the realizations of Bernoulli r.v.s. ξ_i . Nevertheless, when n_t is large the dependence of $I_{t,a}$ on ε_t simplifies, as a consequence of the Law of Large Numbers. To formalize this, we introduce a mild assumption bounding the norm of the user embeddings.

ASSUMPTION 4. $\|X_i\|_2^2 \leq C$ almost surely.

Given that there are a finite number of users, this is generally satisfied. We proceed to show that for large batch sizes, the empirical design matrix converges to a population design matrix, using a standard matrix concentration inequality.

LEMMA 4.3. Suppose Assumption 4 holds and $\varepsilon_t > 0$ for all t . Moreover, suppose that $n_t = \lambda_t n$ for some $\lambda_t \in (0, 1)$ where $\mathbf{1}^\top \lambda = 1$. Then for any $t \leq s \leq T$, for all $\delta > 0$, with probability at least $1 - \delta$ we have that as $n \rightarrow \infty$

$$\left\| \sum_{l=t}^s I_{l,a}(\varepsilon_l) - \sum_{l=t}^s n_l \varepsilon_l I \right\|_{op} = O(n^{1/2} \log(1/\delta))$$

where I is the population design matrix.

$$I \equiv s^{-2} K^{-1} \mathbb{E}_{X_i \sim \mu} [X_i X_i^\top] \quad (24)$$

See Appendix A for the proof.

The population design matrix is more suitable for optimization, as the design matrix no longer depends on ε_t through the realization of Bernoulli r.v.s and instead is simply linear in ε_t . We substitute into (22) to obtain the following approximation for the update for Σ , and we denote the resulting covariance matrices as $\bar{\Sigma}_{t,a}$

$$\bar{\Sigma}_{t,a}^{-1} = \Sigma_{1,a}^{-1} + \sum_{l=1}^{t-1} \varepsilon_l n_l I \quad (25)$$

Let $\bar{J}_1(\varepsilon_{1:T}, \beta_1, \Sigma_1)$ be identical to $J_1(\varepsilon_{1:T}, \beta_1, \Sigma_1)$ in (21), but replacing $\Sigma_{t,a}$ with $\bar{\Sigma}_{t,a}$ for all t, a . Since $\bar{\Sigma}_{t,a}$ is differentiable in $\varepsilon_{1:t-1}$, $\bar{J}_1$ serves as a fully differentiable approximation of Bayesian cumulative regret J . This approximation is also easier to simulate and only requires samples of $Z_t \stackrel{iid}{\sim} N(0, I)$, $X_i \stackrel{iid}{\sim} \mu$.

Algorithm 1 Solve $\min_{\varepsilon_{t:T}} \bar{J}_t(\varepsilon_{t:T}, \beta_t, \Sigma_t)$ via SGD

Require: Current period t , current posterior (β_t, Σ_t) , user embedding samples $\{X_i\}_{i=1}^m$, design matrix I , batch sizes n_t , number of gradient steps L , step size α , feasible domain $\Omega \subseteq [0, 1]^{T-t-1}$.

Ensure: Exploration rates $\varepsilon_{t:T}$

Initialize $\varepsilon_{t:T}^{(0)} \in [0, 1]^{T-t-1}$.

for $k = 0, \dots, L-1$ **do** ▷ L steps of gradient descent

Draw iid $\{Z_s\}_{s=t}^T$ where $Z_s \sim N(0, I)$.

for $s = t, \dots, T$ **do** ▷ Simulate posterior from t to T

$\Sigma_s^{-1} = \Sigma_{t,a}^{-1} + \sum_{l=1}^{s-1} \varepsilon_l^{(k)} n_l I$ ▷ Covariance update

$\beta_{s,a} = \beta_{t,a} + (\Sigma_{t,a} - \Sigma_{s,a})^{1/2} Z_s$ ▷ Mean update

end for

$J(\varepsilon_{t:T}) \leftarrow -\sum_{s=t}^T \frac{1}{m} \sum_{i=1}^m (\varepsilon_s X_i^\top \bar{\beta}_1 + (1 - \varepsilon_s) \max_a X_i^\top \beta_{s,a})$

Compute gradient $\nabla J(\varepsilon_{t:T})$ wrt $\varepsilon_{t:T}$.

$\varepsilon_{t:T}^{(k+1)} \leftarrow \text{Proj}_\Omega(\varepsilon_{t:T}^{(k)} - \alpha \nabla J(\varepsilon_{t:T}))$

end for

We turn to our second main result, which shows that this approximation is accurate as $n \rightarrow \infty$

THEOREM 4.4. Suppose Assumption 4 holds, $n_t = \lambda_t n$ as in Lemma 4.3, and $\varepsilon_t > 0$ for all t . For all $\delta > 0$, with probability at least $1 - \delta$ we have as $n \rightarrow \infty$,

$$|J_1(\varepsilon_{1:T}, \beta_1, \Sigma_1) - \bar{J}_1(\varepsilon_{1:T}, \beta_1, \Sigma_1)| = O(n^{1/4} \log(1/\delta)). \quad (26)$$

See Appendix A for the proof.

The above bound for Bayesian cumulative regret implies that the discrepancy in average Bayesian regret decays at rate $O(n^{-3/4})$ with high probability. Not only is the approximation error sublinear in n , since cumulative regret is known to scale at least $\Omega(n^{1/2})$ [7], this result shows that the approximation error is a much smaller order than regret. Thus, for large sample sizes, $\bar{J}$ can serve as a useful approximation for Bayesian cumulative regret, which is suitable for optimization via gradient-based methods. This result also extends to $J_t(\varepsilon_{t:T}, \beta_t, \Sigma_t)$ and $\bar{J}_t$, which refer to the cumulative regret starting from period t and posterior (β_t, Σ_t) .

5 Exploration via Optimization

Bayesian regret offers a suitable optimization objective as (1) it elicits a differentiable approximation that is accurate under large batches and (2) Monte-Carlo simulation of this approximation can be efficiently generated by simulating $Z_i \sim N(0, I)$ and drawing samples of user embeddings $X_i \sim \mu$.

We proceed to introduce the optimization problem for determining the exploration rates. Note that the optimal reward $n\mathbb{E}_1[r^*(X_i)]$ is not affected by ε , and so minimizing regret is equivalent to maximizing the reward obtained by the policy. This leads to the following stochastic optimization problem: let $Z_t \sim N(0, I)$ be iid standard normal vectors,

$$\begin{aligned} \max_{\varepsilon_{1:T}} \quad & \sum_{t=1}^T n_t \mathbb{E} \left[\varepsilon_t X_i^\top \bar{\beta}_1 + (1 - \varepsilon_t) \max_a X_i^\top \beta_{t,a} \right] \\ \text{s.t.} \quad & \Sigma_{t,a}^{-1} = \Sigma_{1,a}^{-1} + \sum_{l=1}^{t-1} \varepsilon_l n_l I, \quad \forall t = 1, \dots, T \\ & \beta_{t,a} = \beta_{1,a} + (\Sigma_{1,a} - \Sigma_{t,a})^{1/2} Z_t \\ & \varepsilon_t \in [0, 1] \end{aligned} \quad (27)$$

The objective of (27) is differentiable in ε , which motivates the use of stochastic gradient descent (SGD) to optimize the problem (see

Algorithm 2 Model-Predictive Control (MPC)

Require: Prior distribution (β_1, Σ_1) , user embedding samples $\{X_i\}_{i=1}^m$, design matrix $\mathcal{I}$, batch sizes n_t

for $t = 1, \dots, T$ **do**

Solve $\varepsilon_{t:T}^* \leftarrow \arg \min_{\varepsilon_{t:T}} \bar{J}_t(\varepsilon_{t:T}, \beta_t, \Sigma_t)$. ▷ Algorithm 1

Explore with rate ε_t^* in (6).

Collect data $\mathcal{D}_t = \cup_{i=1}^{n_t} \{(X_i, A_i, R_i, \xi_i) : \xi_i = 1\}$

Update posterior $(\beta_{t+1}, \Sigma_{t+1})$ using (12)-(13)

end for

Algorithm 1), where gradients can be efficiently computed via auto-differentiation libraries such as PyTorch [31] or Jax [3]. Although gradient descent is not guaranteed to obtain the global maximizer, as (27) is typically not concave, we find that in practice that variants of SGD, in particular Adam [20], find a good solution.

Solving (27) under the prior (β_1, Σ_1) provides a principled way of obtaining the exploration schedule $\varepsilon_{1:T}$. In addition, we can use the optimization problem as the basis for an *adaptive* exploration strategy. By re-solving the optimization problem (27) at every period t (see Algorithm 2), the platform can adjust the exploration rate during the time horizon, increasing the exploration rate if the policy is learning slower than expected and decreasing it the policy has already converged. This control heuristic, known as model-predictive control (MPC), is widespread in domains such as chemical engineering, aeronautics, etc. [33], but to our knowledge has not yet been applied to the problem of tuning the exploration rate in contextual bandits.

Implementation Details. While optimizing (27) is appealing due to its connection with regret minimization, doing so requires more inputs than simpler strategies. We proceed to discuss these inputs and how they can be set.

- **Samples of user embeddings.** Samples of $\{X_i\}_{i=1}^m$ are important for approximating expected rewards and for estimating the population design matrix $\mathcal{I}$. A simple way to obtain these are to first, set $\varepsilon_1 = 1$ (usually this is optimal) and then use the embeddings from users that arrive in the first batch $t = 1$. Alternatively, the platform could take a random sample of users and query their embeddings.
- **Prior distribution.** While (27) seeks to minimize Bayesian regret under given prior, this prior may be misspecified. For improved performance, the platform can match the prior to the statistics of existing item embeddings. However, we find in experiments we find that even under a simple, misspecified prior $(\beta_1 = 0, \Sigma_1 = I)$, the algorithms perform well.
- **Batch sizes.** The optimization problem requires knowledge of the batch sizes n_t across the horizon. Although it is unrealistic that the platform knows this exactly, these can be estimated from previous patterns of user traffic or impressions. In experiments, we find that our proposed algorithms are robust to errors in n_t , suggesting that n_t does not need to be estimated perfectly to obtain benefits from optimization.
- **Diagonal Approximation.** The optimization problem involves repeated matrix inversions of the posterior covariance matrix, which can be expensive. A more lightweight alternative is to

replace $\Sigma_{t,a}$ and design matrix $\mathcal{I}$ with diagonal approximations for the posterior update, which we do for the experiments.

- **Computational Overhead.** Since Algorithm 2 solves an optimization problem at each batch, it involves more computation than simple heuristics such as epsilon-greedy. Nevertheless, the additional overhead is small; in our experiments we use 10 gradient steps of Adam to optimize (27) (using the diagonal approximation) which takes at most 1 second on a standard laptop CPU.

6 Experiments

In this section, we investigate whether optimizing Bayesian regret produces effective exploration schedules.

Bandit Environment. To benchmark exploration strategies, we use two datasets to construct realistic contextual bandit problems that model cold-start exploration in recommendation systems

- **Movie-Lens 1 Million data (ML-1M)** [15]: consists of 6,000 users, 4,000 movies, and 1M interactions.
- **Netflix Prize data (Netflix)** [2]: we filter users and movies with less than 100 interactions, leaving with 28,604 users, 9,275 movies, and 3M interactions.

In a similar approach to [27], we construct contextual bandit instances by first training a matrix factorization model [21] (embedding dimension $d = 128$) on the data. We train the model to predict the rating, which we use as our reward metric. We use the resulting user embeddings x_i and item embeddings θ_a as the ground-truth reward model $r(x_i, a) = x_i^\top \theta_a$. For each bandit instance, we randomly select a subset of $K \in \{5, 10, 15\}$ items from the total item set as the item set for the instance, denoted as $\mathcal{A}$. We run the contextual bandit policy for the time horizon of T steps, starting from initial item embedding estimates of $\hat{\theta} = \mathbf{0} \in \mathbb{R}^{Kd}$ for the items in $\mathcal{A}$. At each period t , we sample a batch of n_t users uniformly at random from the dataset. For a given user embedding X_i and an item sampled by the policy A_i , we generate noisy rewards from the reward model $R_{i,a} = r(X_i, A_i) + \eta_i$, where $\eta_i \sim N(0, 1)$.

In order to assess the impact of batched feedback, we consider different patterns for the batch arrivals. We let λ_t denote the fraction of users that arrive on period t , and we consider the following arrival patterns:

Increasing: $\lambda_t = [2\%, 18\%, 20\%, 20\%, 20\%, 20\%]$

Spike: $\lambda_t = [5\%, 35\%, 20\%, 20\%, 20\%]$

Constant: $\lambda_t = 10\% \cdot \mathbf{1}_{10}$

These patterns are motivated by user impressions in online platforms, which may experience predictable spikes (e.g. in the beginning of a new item launch) or may increase over time. To generate random arrivals, we draw $n_t \sim \text{Bin}(N, \lambda_t)$, where we consider $N \in \{500, 5000\}$. All the algorithms we benchmark do not know the realizations of n_t , but know N and λ_t . For each choice of K , N , and arrival pattern λ , we run 1000 bandit problems, randomly sampling items and batch sizes n_t each time, and report the regret (3) averaged across all problems.

Bandit Policies. As discussed before, we focus on determining the exploration schedule for uniform exploration policies. Recall

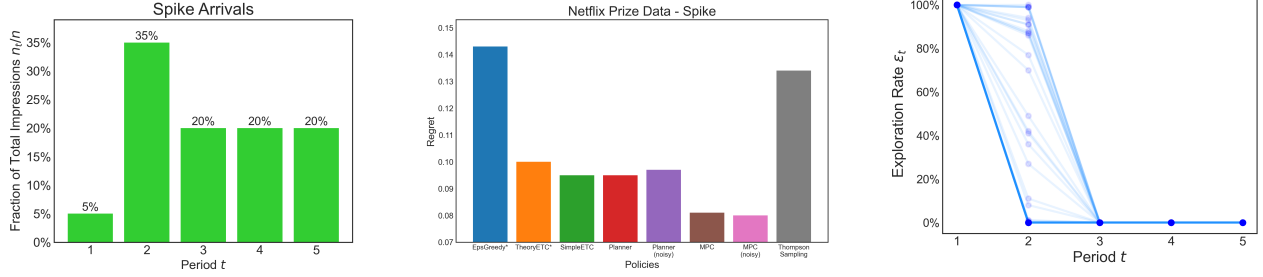


Figure 2: (Left) Fraction of arrivals per period for the spike arrival pattern. (Middle) Average regret across methods for Netflix data with $K = 5$ arms and $N = 500$ samples for the spike arrival pattern. MPC (brown and pink) outperforms other methods in both scenarios (Right) Exploration rates ε_t from MPC for spike arrival patterns, where each line corresponds to one of 100 problem instances. Each line is the trajectory $\varepsilon_{1:T}$ of exploration rates for a problem instance. We see that MPC changes the exploration rates based on the performance of the policy. Often the final exploration rates differ from the initial plan.

that these policies assign an item a to a user X_i according to,

$$\pi_t(a|X_i) = \begin{cases} 1/K & \text{w.p. } \varepsilon_t \\ \mathbf{1}\{a = \arg \max_a X_i^\top \hat{\theta}_{t,a}\} & \text{w.p. } 1 - \varepsilon_t \end{cases}$$

where $\hat{\theta}_{t,a}$ is the Ridge regression estimator of θ_a

$$\hat{\theta}_{t,a} = \arg \min_{\theta \in \mathbb{R}^d} \|R_{t-1,a} - X_{t-1,a}\theta\|_2^2 + \nu \|\theta\|_2^2$$

where $\nu = 1$ is the regularization factor, $X_{t-1,a}$ is the matrix of all user-embeddings assigned a before period t and $R_{t-1,a}$ is the corresponding vector of observed rewards. Note that $\hat{\theta}_{t,a}$ is only estimated from users in the explore group, but we found that the results didn't qualitatively change if we included data from the exploit group as well. The evaluation metric is regret compared to the oracle optimal policy that assigns user i the item $\arg \max_{a \in \mathcal{A}} r(X_i, a)$.

We now discuss the exploration strategies we benchmark for choosing the exploration rates ε_t .

- **Epsilon-Greedy** [EpsGreedy*]: constant exploration $\varepsilon_t = \varepsilon \in [0, 1]$. Performance depends sensitively on ε so we report the regret of *best* $\varepsilon \in [0.05, 0.01, 0.1, 0.5, 1.0]$ for each problem instance (which is different across instances). This is a strong benchmark that is unrealistic to implement in practice, as it requires already knowing the optimal choice of ε .
- **Explore-then-Commit (ETC)** with budget $B = O(d^{1/3}N^{2/3})$ [TheoryETC*]. A theoretical recommendation for ETC is to use a budget of order $O(d^{1/3}N^{2/3})$ [14, 23, 26], and under mild assumptions, this results in a regret scaling of $O(N^{2/3})$. However, setting $B = d^{1/3}N^{2/3}$ tends to explore too much, so instead we report the regret for $B = cd^{1/3}N^{2/3}$ for the *best* constant $c \in [0.05, 0.01, 0.1, 0.5, 1.0]$ (which is different across instances). This benchmark is also unrealistic to use in practice, as it requires already knowing the optimal c .
- **Simple ETC** [SimpleETC]: sets $\varepsilon_1 = 1$ and $\varepsilon_t = 0$ for $t > 1$. Despite its simplicity, it is a commonly used heuristic.

We propose the following exploration strategies based on optimization, and propose variants that test the robustness of these algorithms to errors in the input.

- **Optimization Planner** [Planner]: Obtains the schedule via $\varepsilon_{1:T}^* = \arg \min_{\varepsilon_{1:T}} \bar{J}_1(\varepsilon_{1:T}, \beta_1, \Sigma_1)$ under (mis-specified) prior

$\beta_1 = \mathbf{0}, \Sigma_1 = I$. Obtains samples of X_i from the first batch and uses the batch sizes $\tilde{n}_t = N\lambda_t$ as inputs.

- **Optimization Planner (Noisy)** [Planner (noisy)]: Same as Planner but uses noisy predictions of arrival rates $\hat{n}_t = N\hat{\lambda}_t$ where $\hat{\lambda}_t \sim \text{Dirichlet}_{K\lambda}(\cdot)$ drawn from a Dirichlet distribution with parameter $(K\lambda_1, \dots, K\lambda_T)$.
- **Model-Predictive Control** [MPC]: Runs Algorithm 2 with prior $\beta_1 = \mathbf{0}, \Sigma_1 = I$, uses samples of X_i from the first batch and uses batch sizes $\tilde{n}_t = N\lambda_t$.
- **Model-Predictive Control (Noisy)** [MPC (noisy)]: Same as MPC, but uses noisy predictions of arrival rates $\hat{n}_t = N\hat{\lambda}_t$ where $\hat{\lambda}_t \sim \text{Dirichlet}_{K\lambda}(\cdot)$.

We also benchmark these uniform exploration strategies with a non-uniform exploration policy.

- **Batched Thompson Sampling** [TS] [17, 18]: At each period t , it samples item embeddings from the posterior $\theta_t \sim N(\beta_t, \Sigma_t)$ and assigns items to each user in the batch according to $A_i = \arg \max_{a \in \mathcal{A}} X_i^\top \theta_{t,a}$. It then uses all the batch data to update the posterior mean and covariance.

Results. Table 1 displays the average regret for each policy across different datasets and settings. We observe that the optimization-based algorithms Planner and MPC are able to automatically tailor exploration to the structure of the batches. The exploration schedules consistently match and even outperform SimpleETC and TheoryETC* *uniformly* across patterns, applying the strategy most appropriate to the situation. These algorithms do not overfit either; even under very noisy estimates of the true arrival pattern, both Planner (noisy) and MPC (noisy) achieve only slight worse performance. The consistent performance of this policy is promising for practitioners, as they can use the same algorithm for different settings, instead of having to know beforehand what the appropriate exploration strategy is. We also see that MPC often achieves lower regret than the static Planner, showing that adjusting the exploration rate during the horizon helps correct for any planning errors (see Figure 2).

Finally, we note that surprisingly, the simple uniform exploration policies we consider consistently outperform Thompson Sampling in these batched, short-horizon settings. We find that Thompson

$K = 5$ items												
MovieLens-1M							Netflix					
N samples	500			5000			500			5000		
Arrival λ	Constant	Increasing	Spike	Constant	Increasing	Spike	Constant	Increasing	Spike	Constant	Increasing	Spike
EpsGreedy*	0.278	0.309	0.332	0.145	0.154	0.157	0.121	0.128	0.143	0.065	0.072	0.072
TheoryETC*	0.185	0.156	0.149	0.097	0.048	0.071	0.080	0.093	0.098	0.045	0.039	0.045
SimpleETC	0.139	0.292	0.185	0.074	0.049	0.057	0.105	0.147	0.095	0.040	0.047	0.031
Planner	0.141	0.251	0.184	0.074	0.049	0.057	0.105	0.091	0.095	0.040	0.042	0.031
Planner (noisy)	0.136	0.185	0.187	0.074	0.051	0.057	0.107	0.083	0.097	0.040	0.044	0.031
MPC	0.131	0.164	0.168	0.074	0.045	0.056	0.095	0.084	0.081	0.037	0.033	0.029
MPC (noisy)	0.131	0.160	0.167	0.074	0.037	0.053	0.087	0.088	0.080	0.036	0.031	0.030
TS	0.253	0.287	0.351	0.178	0.189	0.258	0.101	0.113	0.134	0.053	0.061	0.097

$K = 10$ items												
MovieLens-1M							Netflix					
N samples	500			5000			500			5000		
Arrival λ	Constant	Increasing	Spike	Constant	Increasing	Spike	Constant	Increasing	Spike	Constant	Increasing	Spike
EpsGreedy*	0.455	0.496	0.511	0.227	0.232	0.266	0.196	0.226	0.243	0.127	0.132	0.144
TheoryETC*	0.320	0.346	0.292	0.155	0.126	0.117	0.168	0.164	0.167	0.073	0.082	0.082
SimpleETC	0.266	0.625	0.313	0.114	0.141	0.095	0.166	0.259	0.190	0.062	0.145	0.078
Planner	0.265	0.387	0.313	0.114	0.145	0.095	0.165	0.162	0.193	0.062	0.144	0.078
Planner (noisy)	0.255	0.368	0.324	0.114	0.137	0.095	0.164	0.172	0.181	0.062	0.130	0.078
MPC	0.263	0.320	0.292	0.115	0.113	0.093	0.161	0.157	0.154	0.061	0.090	0.078
MPC (noisy)	0.272	0.361	0.291	0.113	0.126	0.084	0.156	0.152	0.160	0.061	0.081	0.076
TS	0.462	0.547	0.651	0.312	0.327	0.404	0.204	0.251	0.247	0.111	0.124	0.197

$K = 15$ items												
MovieLens-1M							Netflix					
N samples	500			5000			500			5000		
Arrival λ	Constant	Increasing	Spike	Constant	Increasing	Spike	Constant	Increasing	Spike	Constant	Increasing	Spike
EpsGreedy*	0.588	0.595	0.623	0.285	0.301	0.346	0.336	0.333	0.346	0.186	0.194	0.219
TheoryETC*	0.433	0.444	0.471	0.185	0.194	0.207	0.237	0.233	0.260	0.090	0.104	0.114
SimpleETC	0.308	0.692	0.523	0.145	0.169	0.124	0.328	0.417	0.310	0.085	0.232	0.099
Planner	0.308	0.402	0.523	0.145	0.165	0.124	0.328	0.307	0.310	0.085	0.232	0.099
Planner (noisy)	0.315	0.425	0.501	0.145	0.167	0.124	0.327	0.302	0.306	0.085	0.230	0.099
MPC	0.309	0.460	0.469	0.145	0.148	0.124	0.286	0.240	0.227	0.085	0.157	0.099
MPC (noisy)	0.308	0.455	0.478	0.145	0.154	0.124	0.296	0.231	0.230	0.082	0.163	0.099
TS	0.525	0.615	0.660	0.278	0.306	0.360	0.290	0.328	0.404	0.209	0.219	0.259

Table 1: Average regret across contextual bandit settings. Each cell is averaged over 1000 bandit problems; standard errors are all close to 0.002. Bolded cells indicates the method(s) with the lowest average regret for each (dataset, horizon, arrival pattern).

Sampling suffers from over-exploration, which has also been observed in other works [8, 16]. One driver of over-exploration is lack of awareness of the time horizon; the algorithm explores until the posterior converges which may be sub-optimal for short time horizons. We find that when time horizon is longer and the arrival pattern is Constant, the performance gap closes. This illustrates that seemingly simple and sub-optimal uniform exploration strategies can be surprisingly effective in settings relevant for practitioners.

7 Conclusion

In this work, we introduce a principled framework for selecting the exploration rates for uniform exploration policies in contextual bandit problems. This framework is based on optimization; selecting the exploration rates ϵ_t to minimize a differentiable approximation of Bayesian regret. We find that optimization is able to calibrate exploration automatically in a wide variety of scenarios

and outperforms existing heuristics. In future work, it would of interest to extend the framework to deep learning models via the NeuralLinear algorithm [35], and the optimization problem could also be extended to include additional constraints or objectives (e.g. Top-K regret).

References

- [1] A. Barraza-Urbina. The exploration-exploitation trade-off in interactive recommender systems. In *Proceedings of the Eleventh ACM Conference on Recommender Systems*, pages 431–435, 2017.
- [2] J. Bennett and S. Lanning. The Netflix Prize. In *Proceedings of KDD Cup and Workshop 2007*, 2007.
- [3] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL <http://github.com/google/jax>.
- [4] E. Che and H. Namkoong. Adaptive experimentation at scale: A computational framework for flexible batches. *arXiv preprint arXiv:2303.11582*, 2023.
- [5] E. Che, D. R. Jiang, H. Namkoong, and J. Wang. Optimization-driven adaptive experimentation. *arXiv preprint arXiv:2408.04570*, 2024.

- [6] M. Chen. Exploration in recommender systems. In *Proceedings of the 15th ACM Conference on Recommender Systems*, pages 551–553, 2021.
- [7] W. Chu, L. Li, L. Reyzi, and R. Schapire. Contextual bandits with linear payoff functions. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 208–214. JMLR Workshop and Conference Proceedings, 2011.
- [8] B. Do, T. Adebisi, and R. Zhang. Epsilon-greedy thompson sampling to bayesian optimization. *Journal of Computing and Information Science in Engineering*, 24(12), 2024.
- [9] H. Esfandiari, A. Karbasi, A. Mehrabian, and V. Mirrokni. Regret bounds for batched bandits. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 7340–7348, 2021.
- [10] A. Foster, D. R. Ivanova, I. Malik, and T. Rainforth. Deep adaptive design: Amortizing sequential bayesian experimental design. In *International conference on machine learning*, pages 3384–3395. PMLR, 2021.
- [11] Z. Gao, Y. Han, Z. Ren, and Z. Zhou. Batched multi-armed bandits problem. *Advances in Neural Information Processing Systems*, 32, 2019.
- [12] A. Garivier, T. Lattimore, and E. Kaufmann. On explore-then-commit strategies. *Advances in Neural Information Processing Systems*, 29, 2016.
- [13] D. Glowacka. Bandit algorithms in recommender systems. In *Proceedings of the 13th ACM conference on recommender systems*, pages 574–575, 2019.
- [14] B. Hao, T. Lattimore, and M. Wang. High-dimensional sparse linear bandits. *Advances in Neural Information Processing Systems*, 33:10753–10763, 2020.
- [15] F. M. Harper and J. A. Konstan. The movielens datasets: History and context. *Acm transactions on interactive intelligent systems (tiis)*, 5(4):1–19, 2015.
- [16] T. Jin, X. Yang, X. Xiao, and P. Xu. Thompson sampling with less exploration is fast and optimal. In *International Conference on Machine Learning*, pages 15239–15261. PMLR, 2023.
- [17] C. Kalkanli and A. Ozgur. Batched thompson sampling. *Advances in Neural Information Processing Systems*, 34:29984–29994, 2021.
- [18] A. Karbasi, V. Mirrokni, and M. Shadravan. Parallelizing thompson sampling. *Advances in Neural Information Processing Systems*, 34:10535–10548, 2021.
- [19] S. Katayama, M. Murooka, and Y. Tazaki. Model predictive control of legged and humanoid robots: models and algorithms. *Advanced Robotics*, 37(5):298–315, 2023.
- [20] D. P. Kingma. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [21] Y. Koren, R. Bell, and C. Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009.
- [22] B. Kouvaritakis and M. Cannon. Model predictive control. *Switzerland: Springer International Publishing*, 38(13-56):7, 2016.
- [23] J. Langford and T. Zhang. The epoch-greedy algorithm for contextual multi-armed bandits. *Advances in neural information processing systems*, 20(1):96–1, 2007.
- [24] T. Lattimore and C. Szepesvári. *Bandit algorithms*. Cambridge University Press, 2020.
- [25] L. Li, W. Chu, J. Langford, and R. E. Schapire. A contextual-bandit approach to personalized news article recommendation. In *Proceedings of the 19th international conference on World wide web*, pages 661–670, 2010.
- [26] W. Li, A. Barik, and J. Honorio. A simple unified framework for high dimensional bandit problems. In *International Conference on Machine Learning*, pages 12619–12655. PMLR, 2022.
- [27] D. Liang and N. Vlassis. Local policy improvement for recommender systems. *arXiv preprint arXiv:2212.11431*, 2022.
- [28] J. Mary, R. Gaudel, and P. Philippe. Bandits warm-up cold recommender systems. *arXiv preprint arXiv:1407.2806*, 2014.
- [29] J. McInerney, B. Lacker, S. Hansen, K. Higley, H. Bouchard, A. Gruson, and R. Mehrotra. Explore, exploit, and explain: personalizing explainable recommendations with bandits. In *Proceedings of the 12th ACM conference on recommender systems*, pages 31–39, 2018.
- [30] H. T. Nguyen, J. Mary, and P. Preux. Cold-start problems in recommendation systems via contextual-bandit algorithms. *arXiv preprint arXiv:1405.7544*, 2014.
- [31] A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, and A. Lerer. Automatic differentiation in pytorch. 2017.
- [32] V. Perchet, P. Rigollet, S. Chassang, and E. Snowberg. Batched bandit problems. 2016.
- [33] S. J. Qin and T. A. Badgwell. An overview of industrial model predictive control technology. In *Alche symposium series*, volume 93, pages 232–256. New York, NY: American Institute of Chemical Engineers, 1971-c2002., 1997.
- [34] T. Rainforth, A. Foster, D. R. Ivanova, and F. Bickford Smith. Modern bayesian experimental design. *Statistical Science*, 39(1):100–114, 2024.
- [35] C. Riquelme, G. Tucker, and J. Snoek. Deep bayesian bandits showdown: An empirical comparison of bayesian deep networks for thompson sampling. *arXiv preprint arXiv:1802.09127*, 2018.
- [36] D. J. Russo, B. Van Roy, A. Kazerouni, I. Osband, Z. Wen, et al. A tutorial on thompson sampling. *Foundations and Trends® in Machine Learning*, 11(1):1–96, 2018.
- [37] A. I. Schein, A. Popescul, L. H. Ungar, and D. M. Pennock. Methods and metrics for cold-start recommendations. In *Proceedings of the 25th annual international ACM SIGIR conference on Research and development in information retrieval*, pages 253–260, 2002.
- [38] N. Silva, T. Silva, H. Werneck, L. Rocha, and A. Pereira. User cold-start problem in multi-armed bandits: when the first recommendations guide the user’s experience. *ACM Transactions on Recommender Systems*, 1(1):1–24, 2023.
- [39] Y. Su, X. Wang, E. Y. Le, L. Liu, Y. Li, H. Lu, B. Lipshitz, S. Badam, L. Heldt, S. Bi, et al. Long-term value of exploration: measurements, findings and algorithms. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*, pages 636–644, 2024.
- [40] R. S. Sutton and A. G. Barto. *Reinforcement learning: An introduction*. MIT press, 2018.
- [41] L. Tang, Y. Jiang, L. Li, and T. Li. Ensemble contextual bandits for personalized recommendation. In *Proceedings of the 8th ACM Conference on Recommender Systems*, pages 73–80, 2014.
- [42] B. van den Akker, O. Jeunen, Y. Li, B. London, Z. Nazari, and D. Parekh. Practical bandits: An industry perspective. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*, pages 1132–1135, 2024.
- [43] J. Vermorel and M. Mohri. Multi-armed bandit algorithms and empirical evaluation. In *European conference on machine learning*, pages 437–448. Springer, 2005.
- [44] X. Wang, Y. Wang, D. Hsu, and Y. Wang. Exploration in interactive personalized music recommendation: a reinforcement learning approach. *ACM Transactions on Multimedia Computing, Communications, and Applications (TOMM)*, 11(1):1–22, 2014.

A Appendix

Proof of Proposition 4.2. First, we can simplify the regret from random exploration

$$\begin{aligned}\mathbb{E}_1[\text{Reg}(X_i, 1/K)] &= \mathbb{E}_1[r^*(X_i)] - \frac{1}{K} \sum_{a=1}^K \mathbb{E}_1[r(X_i, a)] \\ &= \mathbb{E}_1[r^*(X_i)] - \frac{1}{K} \sum_{a=1}^K \mathbb{E}[X_i^\top \mathbb{E}_1[\theta_a]] \\ &= \mathbb{E}_1[r^*(X_i)] - X_i^\top \bar{\beta}_1\end{aligned}$$

For the regret of the greedy policy, we have by the tower property of expectation, Bayesian regret of the greedy policy at period t will be

$$\begin{aligned}\mathbb{E}_1[\text{Reg}(X_i, \hat{\pi}_t)] &= \mathbb{E}_1[r^*(X_i) - \mathbb{E}_{A_i \sim \pi_t(\cdot|X_i)}[r(X_i, A_i)]] \\ &= \mathbb{E}_1[r^*(X_i)] - \mathbb{E}_1[\mathbb{E}_t[\mathbb{E}_{A_i}[r(X_i, A_i)]]] \\ &= \mathbb{E}_1[r^*(X_i)] - \mathbb{E}_1[\max_a X_i^\top \beta_{t,a}]\end{aligned}$$

By Lemma 4.1, we have that conditional on the prior, $\beta_{t,a} \stackrel{d}{=} \beta_{1,a} + (\Sigma_{1,a} - \Sigma_{t,a})^{1/2} Z_t$ where $Z_t \sim N(0, I)$ and $\Sigma_{t,a}$ is defined in (22).

Proof of Lemma 4.3. The first result follows from standard matrix Bernstein inequalities. For simplicity of notation, assume that the measurement variance $s^2 = 1$.

$$\sum_{l=t}^s I_{l,a}(\varepsilon_l) = \sum_{l=t}^s \sum_{i=1}^{n_l} \xi_{l,i} \mathbf{1}\{A_i = a\} X_{l,i} X_{l,i}^\top \equiv \sum_{l=t}^s \sum_{i=1}^{n_l} M_{l,i}$$

where $M_{l,i} = \xi_{l,i} \mathbf{1}\{A_i = a\} X_{l,i} X_{l,i}^\top$. Since, $\|X_i\|_2^2 \leq C$ we have that $\|\xi_{l,i} \mathbf{1}\{A_i = a\} X_{l,i} X_{l,i}^\top\|_{\text{op}} \leq \|X_{l,i} X_{l,i}^\top\|_{\text{op}} \leq C$. Here we include time indices to distinguish between random variables in different time periods. Note that this is a sum of iid random matrices with mean equal to I where $\varepsilon_t I = \varepsilon_t K^{-1} \mathbb{E}[X_i X_i^\top]$. Using the matrix Bernstein inequality

$$\mathbb{P}\left(\left\|\sum_{l=t}^s I_{l,a}(\varepsilon_l) - \sum_{l=t}^s n_l \varepsilon_l I\right\|_{\text{op}} \geq t\right) \leq 2d \exp\left(-\frac{t^2}{2(\sigma^2 + t2C/3)}\right)$$

where

$$\begin{aligned}
\sigma^2 &= \left\| \sum_{l=t}^s \sum_{i=1}^{n_l} \mathbb{E} \left[(M_{t,i} - \varepsilon_t I)^\top (M_{t,i} - \varepsilon_t I) \right] \right\|_{\text{op}} \\
&\leq \sum_{l=t}^s \sum_{i=1}^{n_l} \left\| \mathbb{E} \left[(M_{t,i} - \varepsilon_t I)^\top (M_{t,i} - \varepsilon_t I) \right] \right\|_{\text{op}} \\
&\leq \sum_{l=t}^s \sum_{i=1}^{n_l} 2 \left\| \mathbb{E} \left[M_{t,i}^\top M_{t,i} \right] \right\|_{\text{op}} \\
&\leq \sum_{l=t}^s \sum_{i=1}^{n_l} 2C^2 = n\lambda_{t:s}C^2
\end{aligned}$$

where we use the shorthand $\lambda_{t:s} \equiv \sum_{l=t}^s \lambda_s$. Thus, for any $M > 0$, we have that,

$$\begin{aligned}
&\mathbb{P} \left(\left\| \sum_{l=t}^s I_{l,a}(\varepsilon_l) - \sum_{l=t}^s n_l \varepsilon_l I \right\|_{\text{op}} \geq Mn^{1/2} \right) \\
&\leq 2d \exp \left(-\frac{M^2 n}{2(n\lambda_{t:s}C^2 + n^{1/2}2C/3)} \right)
\end{aligned}$$

Thus, for any $\delta > 0$, we can find an M such that the probability is bounded by δ .

Proof of Theorem 4.4. While the theorem states the bound for $\bar{J}_1$, we prove a more general result for $\bar{J}_t$. First, let $E_n := \sum_{l=t}^s I_{n,l}(\varepsilon_l) - \sum_{l=t}^s n_l \varepsilon_l I$. For clarity, we let $S_n \equiv \sum_{l=t}^s I_{n,l}(\varepsilon_l)$ and $S \equiv \sum_{l=t}^s \lambda_l \varepsilon_l I$ so that $\sum_{l=t}^s I_{n,l}(\varepsilon_l) = nS + E_n$. First, we have that, using the identity $A^{-1} - (A+B)^{-1} = (A+B)^{-1}BA^{-1}$ with $A = (\Sigma_{t,a}^{-1} + nS)$ and $B = E_n$, we have

$$\begin{aligned}
&\left\| \Sigma_{t,a} - (\Sigma_{t,a}^{-1} + nS)^{-1} - (\Sigma_{t,a} - (\Sigma_{t,a}^{-1} + nS + E_n)^{-1}) \right\|_{\text{op}} \\
&= \left\| (\Sigma_{t,a}^{-1} + nS)^{-1} - (\Sigma_{t,a}^{-1} + nS + E_n)^{-1} \right\|_{\text{op}} \\
&= \left\| (\Sigma_{t,a}^{-1} + nS + E_n)^{-1} (E_n) (\Sigma_{t,a}^{-1} + nS) \right\|_{\text{op}} \\
&\leq \left\| (\Sigma_{t,a}^{-1} + nS + E_n)^{-1} \right\|_{\text{op}} \|E_n\|_{\text{op}} \left\| (\Sigma_{t,a}^{-1} + nS) \right\|_{\text{op}}
\end{aligned}$$

We will let $K_t := \left\| (\Sigma_{t,a}^{-1} + nS + E_n)^{-1} \right\|_{\text{op}} \|E_n\|_{\text{op}} \left\| (\Sigma_{t,a}^{-1} + nS) \right\|_{\text{op}}$ in the last line. Note that

$$\begin{aligned}
\left\| (\Sigma_{t,a}^{-1} + nS + E_n)^{-1} \right\|_{\text{op}} &= \lambda_{\max} \left((\Sigma_{t,a}^{-1} + nS + E_n)^{-1} \right) \\
&= \frac{1}{\lambda_{\min} \left((\Sigma_{t,a}^{-1} + nS + E_n) \right)} \\
&\leq \frac{1}{n\lambda_{\min}(S) + \lambda_{\min}(E_n)} \\
&\leq \frac{1}{n\lambda_{\min}(S) - \|E_n\|_{\text{op}}}
\end{aligned}$$

and by a similar argument $\left\| (\Sigma_{t,a}^{-1} + nS) \right\|_{\text{op}} \leq (n\lambda_{\min}(S))^{-1}$. This gives us an upper bound of

$$\begin{aligned}
&\left\| (\Sigma_{t,a}^{-1} + nS)^{-1} - (\Sigma_{t,a}^{-1} + nS + E_n)^{-1} \right\|_{\text{op}} \\
&\leq \frac{\|E_n\|_{\text{op}}}{n\lambda_{\min}(S) - \|E_n\|_{\text{op}}} \frac{1}{n\lambda_{\min}(S)}
\end{aligned}$$

Thus, we have that for any $M > 0$, there exists N such that for all $n \geq N$,

$$\begin{aligned}
&\mathbb{P} \left(\left\| (\Sigma_{t,a}^{-1} + nS)^{-1} - (\Sigma_{t,a}^{-1} + nS + E_n)^{-1} \right\|_{\text{op}} > Mn^{-3/2} \right) \\
&\leq \mathbb{P} \left(\frac{\|E_n\|_{\text{op}}}{n\lambda_{\min}(S) - \|E_n\|_{\text{op}}} \frac{1}{n\lambda_{\min}(S)} > Mn^{-3/2} \right) \\
&= \mathbb{P} \left(\|E_n\|_{\text{op}} > M\lambda_{\min}(S)n^{-1/2}(n\lambda_{\min}(S) - \|E_n\|_{\text{op}}) \right) \\
&= \mathbb{P} \left(\|E_n\|_{\text{op}}(1 + M\lambda_{\min}(S)n^{-1/2}) > M\lambda_{\min}(S)n^{-1/2} \right) \\
&= O(e^{-M^2})
\end{aligned}$$

Recall that J_t is defined as,

$$J_t = n\mathbb{E}_t[r^*(X_i)] - \sum_{s=t}^T n_s \left(\varepsilon_s \mathbb{E}[X_i]^\top \bar{\beta}_t + (1 - \varepsilon_s) \mathbb{E}_t[\max_a X_i^\top \beta_{s,a}] \right)$$

$$\text{where } \beta_{s,a} = \beta_{t,a} + (\Sigma_{t,a} - \Sigma_{s,a})^{1/2} Z_{t,a}$$

and $\bar{J}_t$ is identical with the replacement of $\beta_{s,a}$ by $\bar{\beta}_{s,a}$ where

$$\bar{\beta}_{s,a} = \beta_{t,a} + (\Sigma_{t,a} - \bar{\Sigma}_{s,a})^{1/2} Z_{t,a}$$

Note then that

$$J_t - \bar{J}_t = \sum_{s=t}^T n_s (1 - \varepsilon_s) \left(\mathbb{E}_t[\max_a X_i^\top \beta_{s,a}] - \mathbb{E}_t[\max_a X_i^\top \bar{\beta}_{s,a}] \right)$$

So one need only to bound the difference in $\mathbb{E}_t[\max_a X_i^\top \beta_{s,a}] - \mathbb{E}_t[\max_a X_i^\top \bar{\beta}_{s,a}]$. First, we have that

$$X_i^\top \beta_{s,a} \sim N(0, X_i^\top (\Sigma_{t,a} - \Sigma_{s,a}) X_i)$$

$$X_i^\top \bar{\beta}_{s,a} \sim N(0, X_i^\top (\Sigma_{t,a} - \bar{\Sigma}_{s,a}) X_i)$$

Thus, for $z_a \sim N(0, 1)$ distributed iid we have

$$\begin{aligned}
&|\mathbb{E}_t[\max_a X_i^\top \beta_{s,a}] - \mathbb{E}_t[\max_a X_i^\top \bar{\beta}_{s,a}]| \\
&= |\mathbb{E}_{X_i} \mathbb{E}_z [\max_a \sqrt{X_i^\top (\Sigma_{t,a} - \Sigma_{s,a}) X_i} z_a - \max_a \sqrt{X_i^\top (\Sigma_{t,a} - \bar{\Sigma}_{s,a}) X_i} z_a]| \\
&\leq \mathbb{E}_{X_i} \mathbb{E}_z [|\max_a \sqrt{X_i^\top (\Sigma_{t,a} - \Sigma_{s,a}) X_i} z_a - \max_a \sqrt{X_i^\top (\Sigma_{t,a} - \bar{\Sigma}_{s,a}) X_i} z_a|] \\
&= \mathbb{E}_{X_i} \mathbb{E}_z [\max_a |\sqrt{X_i^\top (\Sigma_{t,a} - \Sigma_{s,a}) X_i} z_a - \sqrt{X_i^\top (\Sigma_{t,a} - \bar{\Sigma}_{s,a}) X_i} z_a|] \\
&\leq \mathbb{E}_{X_i} \mathbb{E}_z [\max_a |\sqrt{X_i^\top (\Sigma_{t,a} - \Sigma_{s,a}) X_i} - \sqrt{X_i^\top (\Sigma_{t,a} - \bar{\Sigma}_{s,a}) X_i}| |z_a|] \\
&\leq \mathbb{E}_{X_i} \mathbb{E}_z [\max_a |\sqrt{X_i^\top (\Sigma_{t,a} - \Sigma_{s,a}) X_i} - \sqrt{X_i^\top (\Sigma_{t,a} - \bar{\Sigma}_{s,a}) X_i}| |z_a|] \\
&= \mathbb{E}_{X_i} \mathbb{E}_z [\max_a |\sqrt{X_i^\top (\bar{\Sigma}_{s,a} - \Sigma_{s,a}) X_i}| |z_a|]
\end{aligned}$$

where we use the inequality $|\max_a x_a - \max_a y_a| \leq \max_a |x_a - y_a|$ and $|\sqrt{a} - \sqrt{b}| \leq \sqrt{|a - b|}$. Finally, we have that

$$\begin{aligned}
&\mathbb{E}_{X_i} \mathbb{E}_t [\max_a |\sqrt{X_i^\top (\bar{\Sigma}_{s,a} - \Sigma_{s,a}) X_i}| |z_a|] \\
&\leq \mathbb{E}_{X_i} \max_a |\sqrt{X_i^\top (\bar{\Sigma}_{s,a} - \Sigma_{s,a}) X_i}| \cdot \mathbb{E}_z [\max_a |z_a|] \\
&\leq \mathbb{E}_{X_i} \max_a |\sqrt{\|\bar{\Sigma}_{s,a} - \Sigma_{s,a}\|_{\text{op}} \|X_i\|_2^2}| \cdot \mathbb{E}_z [\max_a |z_a|] \\
&\leq \sqrt{C} \sqrt{\max_a \|\bar{\Sigma}_{s,a} - \Sigma_{s,a}\|_{\text{op}}} \cdot \sqrt{2 \log 2K}
\end{aligned}$$

where we used a standard bound for $\mathbb{E}_z[\max_a |z_a|] \leq \sqrt{2 \log 2K}$. This leaves us with

$$|J_t - \bar{J}_t| \leq \sum_{s=t}^T n_s (1 - \varepsilon_s) \sqrt{2C \log 2K} \sqrt{\max_a \|\bar{\Sigma}_{s,a} - \Sigma_{s,a}\|_{\text{op}}}$$

By applying the probability bound in Lemma we have that for any $M > 0$,

$$\begin{aligned}
 & \mathbb{P}\left(|J_t - \bar{J}_t| > Mn^{1/4}\right) \\
 & \leq \mathbb{P}\left(n\sqrt{2C \log 2K} \sqrt{\max_a \|\bar{\Sigma}_{s,a} - \Sigma_{s,a}\|_{\text{op}}} > Mn^{1/4}\right) \\
 & = \mathbb{P}\left(\max_a \|\bar{\Sigma}_{s,a} - \Sigma_{s,a}\|_{\text{op}} > (2C \log 2K)^{-1} Mn^{-3/2}\right) \\
 & \leq \sum_a \mathbb{P}\left(\|\bar{\Sigma}_{s,a} - \Sigma_{s,a}\|_{\text{op}} > (2C \log 2K)^{-1} Mn^{-3/2}\right) \\
 & = O(Ke^{-M^2})
 \end{aligned}$$

By setting $M = O(\log(1/\delta))$, we have the desired result.